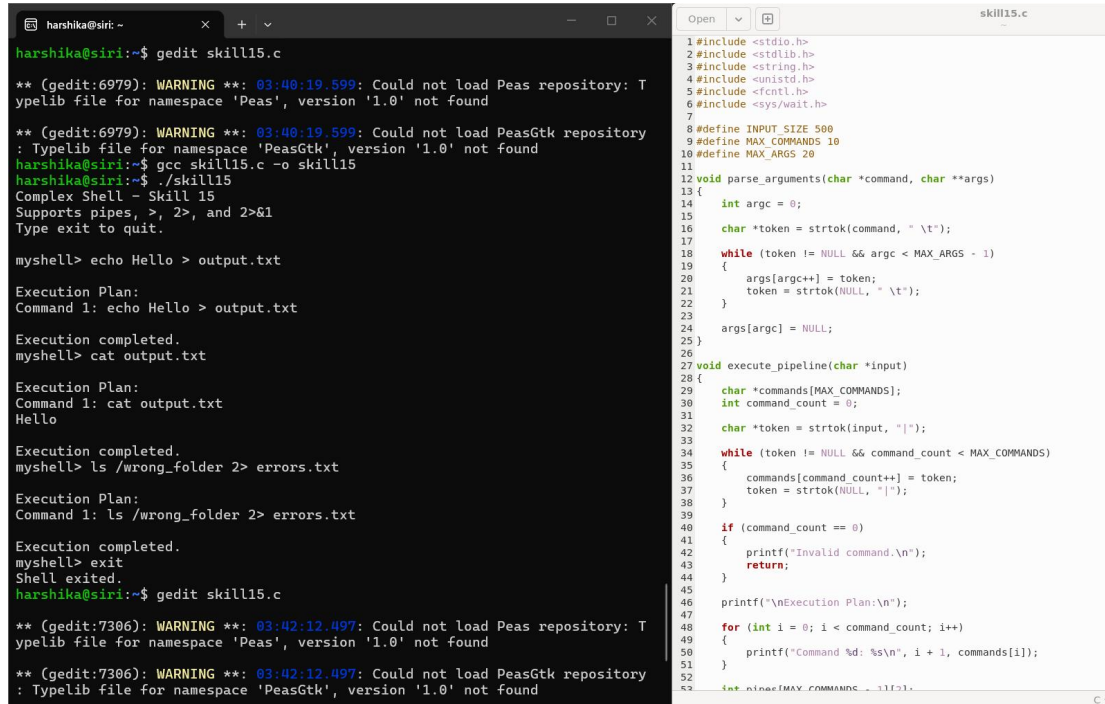


Skill - 15

1. Output redirection



The image shows a terminal window on the left and a code editor on the right. The terminal window displays the execution of a C program named `skill15.c`. The user runs `gcc skill15.c -o skill15` and then `./skill15`. The program prompts for input and output redirection. The user enters `echo Hello > output.txt` and `cat output.txt`, which results in the output `Hello` being redirected to `output.txt`. The user then enters `ls /wrong_folder 2> errors.txt`, which results in an error message being redirected to `errors.txt`. The terminal window also shows warnings from Gedit about Peas repository and Typelib file issues.

```
harshika@siri:~$ gedit skill15.c

** (gedit:6979): WARNING **: 03:40:19.599: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:6979): WARNING **: 03:40:19.599: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
harshika@siri:~$ gcc skill15.c -o skill15
harshika@siri:~$ ./skill15
Complex Shell - Skill 15
Supports pipes, >, 2>, and 2>&1
Type exit to quit.

myshell> echo Hello > output.txt

Execution Plan:
Command 1: echo Hello > output.txt

Execution completed.
myshell> cat output.txt

Execution Plan:
Command 1: cat output.txt
Hello

Execution completed.
myshell> ls /wrong_folder 2> errors.txt

Execution Plan:
Command 1: ls /wrong_folder 2> errors.txt

Execution completed.
myshell> exit
Shell exited.
harshika@siri:~$ gedit skill15.c

** (gedit:7306): WARNING **: 03:42:12.497: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:7306): WARNING **: 03:42:12.497: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <fcntl.h>
6 #include <sys/wait.h>
7
8 #define INPUT_SIZE 500
9 #define MAX_COMMANDS 10
10 #define MAX_ARGS 20
11
12 void parse_arguments(char *command, char **args)
13 {
14     int argc = 0;
15
16     char *token = strtok(command, " \t");
17
18     while (token != NULL && argc < MAX_ARGS - 1)
19     {
20         args[argc++] = token;
21         token = strtok(NULL, " \t");
22     }
23
24     args[argc] = NULL;
25 }
26
27 void execute_pipeline(char *input)
28 {
29     char *commands[MAX_COMMANDS];
30     int command_count = 0;
31
32     char *token = strtok(input, "|");
33
34     while (token != NULL && command_count < MAX_COMMANDS)
35     {
36         commands[command_count++] = token;
37         token = strtok(NULL, "|");
38     }
39
40     if (command_count == 0)
41     {
42         printf("Invalid command.\n");
43         return;
44     }
45
46     printf("\nExecution Plan:\n");
47
48     for (int i = 0; i < command_count; i++)
49     {
50         printf("Command %d: %s\n", i + 1, commands[i]);
51     }
52
53     int pipefd[MAX_COMMANDS + 1];
```